

Java - Threads and Concurrency

Amir Kirsh and Hiddai Oren

© All rights reserved.
Materials are for the sole use of AT&T Upskill course participants, Sep-Oct 2026. Any other use is forbidden

Agenda

- Intro
- Synchronization
- ThreadPool
- Thread Local Variables
- Fork-Join (extra materials)

What are Threads?

- A thread is a lightweight sub-process, a smallest unit of processing.
- Allows multiple tasks to be executed concurrently with a single program

- Examples?

Creating a thread in Java

- Extending the Thread class
- Implementing the Runnable interface
- Using the Callable interface and Future

Creating a thread in Java - Thread class

A thread can be created by extending the Thread class and overriding the ‘run’ method.

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println( "Thread is running" );
    }
}

MyThread t = new MyThread();
t.start();
```

Creating a thread in Java - Thread class

A thread can be created by extending the Thread class and overriding the ‘run’ method.

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println( "Thread is running" );
    }
}

MyThread t = new MyThread();
t.start();
```

- Pros: Simple to implement
- Cons: Not flexible, Thread and task are coupled

Creating a thread in Java - Runnable interface

A thread can be created by implementing the Runnable interface

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable is running");
    }
}

Thread t = new Thread(new MyRunnable());

t.start();
```

Creating a thread in Java - Runnable interface

A thread can be created by implementing the Runnable interface

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable is running");
    }
}

Thread t = new Thread(new MyRunnable());

t.start();
```

Creating a thread in Java - Runnable interface

A thread can be created by implementing the Runnable interface

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable is running");
    }
}

Thread t = new Thread(new MyRunnable());

t.start();
```

- Pros: Solving the flexibility problem, Decouples task from thread
- Cons: A bit more complex

Creating a thread in Java - Callable & Future

Resembles Runnable, but can throw exceptions and return result

```
class MyCallable implements Callable<String> {

    public String call() {
        return "Callable result";
    }
}
```

Functional Interface

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

A functional interface is an interface with **exactly one abstract method**.

A lambda function can implement a functional interface if:

- It has matching parameters and return type
- It has compatible checked exceptions (it can also not have any checked exceptions - as is the case in regular implementing methods)

Read more:

https://docs.oracle.com/javase/8/docs/api/java/lang/FunctionalInterface.html

Creating a thread in Java - Callable & Future

```
public static void main(String[] args) {
    ExecutorService executor = Executors.newSingleThreadExecutor();
    Callable<String> myCallable = () ->  "Test";
    // non-blocking asynchronous submission of the callable to the executor
    Future<String> future = executor.submit(myCallable);
    try {
       // blocking with a timeout
        String result = future.get( 100, TimeUnit.MILLISECONDS);
        System.out.println(result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();  // Better to replace with appropriate logs
    } catch (TimeoutException e){
        // Program fallback logic + appropriate logs
    }
    executor.shutdown();
}
```

Creating a thread in Java - Callable & Future

```
public static void main(String[] args) {
    ExecutorService executor = Executors.newSingleThreadExecutor();
    Callable<String> myCallable = () -> "Test";
    // non-blocking asynchronous submission of the callable to the executor
    Future<String> future = executor.submit(myCallable);
    try {
        // blocking with a timeout
        String result = future.get( 100, TimeUnit.MILLISECONDS);
        System.out.println(result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();  // Better to replace with appropriate logs
    } catch (TimeoutException e){
        // Program fallback logic + appropriate logs
    }
    executor.shutdown();
}
```

Creating a thread in Java - Callable & Future

```
public static void main(String[] args) {
    ExecutorService executor = Executors.newSingleThreadExecutor();
    Callable<String> myCallable = () -> "Test";
    // non-blocking asynchronous submission of the callable to the executor
    Future<String> future = executor.submit(myCallable);
    try {
        // blocking with a timeout
        String result = future.get( 100, TimeUnit.MILLISECONDS);
        System.out.println(result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();  // Better to replace with appropriate logs
    } catch (TimeoutException e){
        // Program fallback logic + appropriate logs
    }
    executor.shutdown();
}
```

Creating a thread in Java - Sanity Test

```
public static void main(String[] args) {
    Thread t = new Thread(() -> System.out.println("yo!"));
    t.run();
    System.out.println("main");
}


// what will happen when running this main?
```

Setting thread parameters

```
public Thread(ThreadGroup group,
              Runnable target,
              String name,
              long stackSize)
```

https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#<init>(java.lang.ThreadGroup,java.lang.Runnable,java.lang.String,long)

ThreadGroup - ?

https://stackoverflow.com/questions/1649133/what-is-the-benefit-of-threadgroup-in-java-over-creating-separate-threads

https://wiki.sei.cmu.edu/confluence/display/java/THI01-J.+Do+not+invoke+ThreadGroup+methods

Stack size - relevant? read the documentation, validate, tune!

See also - setting thread priority:

https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#setPriority(int)

Before Daemon Thread

```
public static void main(String[] args) {
  Thread t = new Thread( () -> {
    try{
      Thread.sleep(1000);
    }catch(InterruptedException e ){
      System.out.println("Thread Interrupted!");
    }
    System.out.println("yo!");
  });
  t.start();
  System.out.println("main");

}
// what will be printed?
```

Before Daemon Thread

```
public static void main(String[] args) {
  Thread t = new Thread( () -> {
    try{
      Thread.sleep(1000);
    }catch(InterruptedException e ){
      System.out.println("Thread Interrupted!");
    }
    System.out.println("yo!");
  });
  t.start();
  System.out.println("main");

}
// what will be printed?
```

Before Daemon Thread - nicer syntax

```
public static void main(String[] args) {
    Thread t = new Thread( () -> {
        // com.google.common.util.concurrent.Uninterruptibles
        Uninterruptibles.sleepUninterruptibly( 1, TimeUnit.SECONDS);
        System.out.println( "yo!");
        });
    t.start();
    System.out.println( "main");
}

// what will be printed now?
```

Daemon Thread

```
public static void main(String[] args) {
    Thread t = new Thread( () -> {
        // com.google.common.util.concurrent.Uninterruptibles
        Uninterruptibles.sleepUninterruptibly( 1, TimeUnit.SECONDS);
        System.out.println( "yo!");
    });
    t.setDaemon(true);
    t.start();
    // Uninterruptibles.sleepUninterruptibly(2, TimeUnit.SECONDS);
    System.out.println( "main");
}

// what will be printed now?
```

Agenda

- Intro
- Synchronization
- ThreadPool
- Thread Local Variables
- Fork-Join (extra materials)

Synchronization

- Prevents memory consistency errors
- Synchronized method

```
class Counter {
   private int count = 0;
   public synchronized void increment() {
       count++;
   }
   public int getCount() {
       return count;
   }
}
```

- When a thread calls the synchronized method, it acquires the intrinsic lock. No other thread can call the method

Synchronization

A “crash course” on data corruption and ‘mutual exclusive’:

```
public class SynchronizedCounter {
  private int c = 0;
  public synchronized void increment() { c++; }
  public synchronized void decrement() { c--; }
  public synchronized int value() { return c; }
}
```

The synchronized keyword on a method means that if this is already locked anywhere (on this method or elsewhere) by another thread, we need to wait till this is unlocked before we can enter



Synchronization

- In java every object has an **intrinsic** lock (known as monitor)
- Synchronized block

```
class Counter {
   private int count = 0;
   private final Object lock = new Object();
   public void increment() {
       synchronized (lock) {
           count++;
       }
   }
   public int getCount() { return count; }
}
```

- The thread acquires a specific lock (lock)
- Synchronization is only for the critical section

Synchronization

```
public class TwoCounters {
    private long c1 = 0, c2 = 0;
    private Object lock1 = new Object();
    private Object lock2 = new Object();
    public void inc1() {
        synchronized(lock1) {
            c1++;
        }
    }
    public void inc2() {
        synchronized(lock2) {
            c2++;
        }
    }
}
```

Synchronization

```
public void addName(String name) {
  synchronized(this) {
      lastName = name;
      nameCount++;
      nameList.add(name);
  }
  // do things that don't require synchronization
}
```

When synchronizing a block, key for the locking should be supplied (usually would be this).

The advantage of not synchronizing the entire method is efficiency

Synchronization

What does synchronize on a static method mean?

```
public class SynchronizedCounter {
  public static synchronized int getValue() {
  // do something that need to be synchronized
  }
}
```

The lock is the class object representing this class.

Note: synchronized on static method should be very careful.

Atomic Variables

Allows atomic operations on a variable without synchronization.
Usually should be more efficient.

Implementation is Hardware and OS dependent but may use Compare and Swap (CAS).

See: https://docs.oracle.com/javase/tutorial/essential/concurrency/atomicvars.html
https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/atomic/package-summary.html

Atomic Variables

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicCounter {
  private AtomicInteger c = new AtomicInteger(0);
  public void increment() { c.incrementAndGet(); } //No synchronization
  public void decrement() { c.decrementAndGet(); }
  public int value() { return c.get(); }
}
```

Additional locks and Sync Utils

java.util.concurrent.locks

- LockSupport park, unpark, parkUntil
- ReentrantLock + Condition
  (resembles wait notify, with: await, signal, see:
  https://stackoverflow.com/questions/10395571/condition-vs-wait-notify-mechanism)
- ReentrantReadWriteLock vs. StampedLock -- both for Read-Write
  (see: https://stackoverflow.com/questions/28918004/readwritelock-vs-stampedlock)

java.util.concurrent

- Semaphore - allow N acquires before blocking
- CountDownLatch, CyclicBarrier, Phaser - wait for completion of N activities
  (see: https://stackoverflow.com/questions/184147/countdownlatch-vs-semaphore)
- Exchanger - exchange objects between two threads in a thread safe way

Agenda

- Intro
- Synchronization
- ThreadPool
- Thread Local Variables
- Fork-Join (extra materials)

Thread Pools - Executor Service

- Managing a pool of worker threads and provides methods to submit tasks for execution.
- Simplifies the process of managing threads - in compare to manual thread creation.
- Efficient reuse of threads, reducing creation time/ destruction time.
- Java interface in the java.util.concurrent package.

Avoid using raw thread!!! Use executor and thread pools.

Example - Fixed Thread Pool

```
public class FixedThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executorService = Executors.newFixedThreadPool( 4);
        for (int i = 0; i < 10; i++) {
            final int taskId = i;
            executorService.submit(() -> {
                System.out.println( "Task "+taskId+"is running thread "
                                    +Thread.currentThread().getName());
                try { Thread.sleep(2000); } // Simulate thread work
                catch (InterruptedException e) {
                   Thread.currentThread().interrupt();
                }
                System.out.println( "Task " + taskId + " completed");
            });
        }
        executorService.shutdown();
    }
}
```

Exercise - ThreadPool

Read from a file line by line and send each line as a task to a ThreadPool.

Each line should have userID and userName in the following format:

userid=<userID>, username=<userName>

Example input:

userid=123, username=Momo

Create a Map of userID to userName.

Optional (When the base functionality works):

1. in case where a userID already exists:
   a. If the name is the same - ignore.
   b. If the name is different, keep the one that is already in the map and log the event.
2. Lines that do not follow the format should be copied to an Errors collection (of lines + appropriate logs).

Agenda

- Intro
- Synchronization
- ThreadPool
- Thread Local Variables
- Fork-Join (extra materials)

Thread Local Variables

- `ThreadLocal` provides variables that each thread accessing them has its own independent initialized copy.
- Useful for Storing per-thread context information -
  - User session management
  - Database connection management
- No need for explicit synchronization when accessing (reducing the complexity of the concurrent code)

Thread Local Variables - Toy example

```
public class ThreadId {
    // Atomic integer containing the next thread ID to be assigned
    private static final AtomicInteger nextId = new AtomicInteger(0);

    // Thread local variable containing each thread's ID
    private static final ThreadLocal<Integer> threadId =
        new ThreadLocal<Integer>() {
            @Override
            protected Integer initialValue() {
                return nextId.getAndIncrement();
            }
        };

    // Returns the current thread's unique ID, assigning it if necessary
    public static int get() {
        return threadId.get();
    }
}
```

Thread Local Variables - Toy example

```
public class ThreadId {
    // Atomic integer containing the next thread ID to be assigned
    private static final AtomicInteger nextId = new AtomicInteger(0);

    // Thread local variable containing each thread's ID
    private static final ThreadLocal<Integer> threadId =
        new ThreadLocal<Integer>() {
            @Override
            protected Integer initialValue() {
                return nextId.getAndIncrement();
            }
        };

    // Returns the current thread's unique ID, assigning it if necessary
    public static int get() {
        return threadId.get();
    }
}
```

Anonymous subclass!

Thread Local Variables - Simpler syntax

```
public class ThreadId {
    // Atomic integer containing the next thread ID to be assigned
    private static final AtomicInteger nextId = new AtomicInteger(0);

    // Thread local variable containing each thread's ID
    private static final ThreadLocal<Integer> threadId =
        ThreadLocal.withInitial( ()  -> nextId.getAndIncrement() );

    // Returns the current thread's unique ID, assigning it if necessary
    public static int get() {
        return threadId.get();
    }
}
```

That concludes this chapter

Thank You